Using Machine Learning Approaches for Predicting Time of Death of Human Postmortem Samples Based on Transcriptomic Data

Olivia Liau[1], Tillie Slosser[2], Ivan Betancourt[3], Qiaochu Liu[4], Chun-Kit Ngan[4,*], Chen Fu[5,*], Ryan W. Logan[5] and Nitya Phani Santosh Oruganty[4]

[1]*University of Southern California, 3551 Trousdale Pkwy, Los Angeles, CA, U.S.A.*

[2]*Smith College, 10 Elm Street, Northampton, MA, U.S.A.*

[3]*Amherst College, 220 South Pleasant Street, Amherst, MA, U.S.A.*

[4]*Worcester Polytechnic Institute, 100 Institute Road, Worcester, MA, U.S.A.*

[5]*UMass Chan Medical School, 366 Plantation Street, Worcester, MA, U.S.A.*

oliau@usc.edu, tillie@slosser.net, ibetancourt26@amherst.edu, {qliu4, cngan, noruganty}@wpi.edu, {Chen.Fu, Ryan.Logan}@umassmed.edu

Keywords: Transcriptomics, Gene Expression, Circadian Rhythms, Machine Learning, AutoEncoder, Dimensionality Reduction, Model Optimization.

Abstract: In this work, we introduce a machine learning (ML) pipeline that predicts the time of death (TOD) of a subject from gene expression (GE) profiles, addressing a critical gap in genomic research where TOD data are scarce. Our contributions are fourfold: (1) a data-driven, clinically domain-guided pipeline that learns temporal GE patterns for TOD prediction; (2) a two-stage dimensionality reduction approach combining (I) AutoEncoders and (II) ISOMAP for BA11 & PCA for BA47, preserving the temporal sequence of the data while incorporating domain knowledge and obviating exhaustive searches for optimal circadian gene sequences; (3) systematic training and hyperparameter tuning of 16 ML regressors—including five single, eight ensemble, and three deep learning models—to identify the most effective ML model (i.e., ExtraTrees Regressor for BA11 and AdaBoost Regressor for BA47); and (4) a comprehensive evaluation on 146 subjects, examining 235 circadian gene's expression patterns per subject across five performance metrics. Our method surpasses both non-temporal-encoding-based and temporal-encoding-based models, achieving hourly scaled MAEs of 0.839 (BA11) and 1.227 (BA47), with corresponding MSE and RMSE values of 1.013/1.006 and 2.153/1.467, respectively. Consequently, TOD predictions fall within a one hour error margin for BA11 and a two hour margin for BA47.

1 INTRODUCTION

Circadian rhythms (Anafi et al., 2017), the natural 24-hour cycles that regulate various physiological processes, have significant implications for human health, including psychiatric disorders, cancer, and other diseases. Disruptions to circadian rhythms, often driven by factors such as shift work, irregular sleep patterns, or genetic variants in circadian clock genes, can profoundly affect mental health. For instance, abnormalities in circadian regulation have been implicated in mood disorders, including depression, bipolar disorder, and schizophrenia, where circadian gene polymorphisms have been linked to altered sleep-wake cycles and mood regulation (Landgraf et al., 2014; McClung, 2013). Circadian disruption may exacerbate psychiatric symptoms by affecting neurobiological pathways involving neurotransmitters including serotonin and dopamine, as well as neurotrophic factors such as brain-derived neurotrophic factor (BDNF), which are crucial for mood stabilization and cognitive function (Castren and Antila, 2017).

Circadian rhythms are also essential for regulating cell proliferation, apoptosis, and DNA repair, which are key in cancer prevention, with disruptions increasing the risk of cancers including breast cancer, prostate cancer, and colorectal cancer (Blask, 2009; Chen-Goodspeed and Lee, 2007). Circadian misalignment also affects metabolic and cardiovascular health, contributing to diabetes, obesity, and hyper-

*Corresponding Authors.

tension (Marcheva et al., 2010). These findings highlight the importance of incorporating circadian biology into disease prevention and treatment strategies.

As a classical approach to understanding the etiology of these diseases, efforts to identify differentially expressed genes from bulk or single-cell RNASeq data have been made over the past decades (Zhang et al., 2019). Similar to its significant contributions to psychiatric disorders, cancer development, and metabolic and cardiovascular diseases, the circadian component profoundly influences gene expression changes. In human, 1000-3000 genes were reported to exhibit changes in expression over the circadian (Zeitgeber Time, ZT) cycle (Moller-Levet et al., 2013; Zhang et al., 2014). Notably, time of death (TOD) has been shown to account for a large proportion of variance in gene expression (GE), underscoring the importance of circadian timing in biological processes (Li et al., 2018; Logan and McClung, 2017). However, the TOD information is often not available to transcriptome and proteomic datasets, including GE Omnibus (GEO) (National Center for Biotechnology Information, 2024) and Genotype-Tissue Expression (GTEx) (Broad Institute of MIT and Harvard, 2024).

To address the above issue, traditional machine learning (ML) approaches, namely non-temporal-encoding-based methods, including single-based models (e.g., Linear Regressor (Liu et al., 2019)), ensemble-based models (e.g., Gradient Boosting Regressor (Li et al., 2019)), and deep learning-based models (e.g., Multilayer Perceptron Neural Network (Eetemadi and Tagkopoulos, 2018), have been devoted to estimate TODs using transcriptomic data from human biobanks (Arnold et al., 2021). However, these models are subject to a critical limitation. That is, they are impractical due to the inherent complexities of circadian GE. Specifically, the sinusoidal nature of circadian genes poses a significant challenge, as identical GE levels can be observed at two distinct TODs. This ambiguity arises from the failure of those ML models to account for the sequential nature of circadian GE, thereby limiting their ability to predict TODs with precision.

An alternative approach involves leveraging temporal-encoding-based methods, including Long Short-Term Memory (LSTM) Networks (Wang et al., 2019) and Convolutional Neural Networks (CNNs) (Yasinska-Damri et al., 2023). LSTM networks, a variant of Recurrent Neural Network (RNN) architecture, are particularly adept at capturing long-term dependencies, making them well-suited for sequence prediction tasks. Designed to handle sequential data, LSTMs effectively model temporal relationships and dependencies within data. The incorporation of feedback connections enables LSTMs to process entire sequences, rather than isolated data points, thereby enhancing their ability to understand and predict patterns in sequential data. However, LSTM networks are not designed for capturing spatial hierarchies and local patterns in data. Notably, in our specific case, we have 235 circadian GEs (Chen et al., 2016) for each TOD within an observable time window.

CNNs are another type of deep learning-based models that has also been explored for predicting TODs based on postmortem GE profiles. Although CNNs are traditionally used for image data, their ability to capture spatial patterns has been adapted to temporal and sequential data, including GE over time. A recent study applied CNNs to predict TODs using transcriptomic data from postmortem tissues. CNNs are leveraged to identify and learn GE patterns that are temporally correlated with TODs, allowing for more accurate prediction compared to traditional ML models. CNNs can extract complex, high-dimensional features from GE data, which are crucial for identifying time-sensitive biomarkers of TODs (Yang et al., 2021). However, due to the requirements of building large two-dimensional (2D) input matrices to CNNs, searching for the optimal order of those numerous GE patterns across multiple circadian genes within an observable time window to extract the most important features is very challenging and time-consuming that results in inaccurate predictions on TODs.

To bridge the above shortcomings, we propose a transparent, scalable framework to support the TOD prediction using GE patterns over time. Specifically, our contributions to this work are four-fold:

1. We design a Data-driven, Clinically domain-guided (DC) pipeline to learn GE patterns over time to perform TOD predictions.
2. We develop a two-stage dimensionality reduction (DR) approach which combines (I) AutoEncoders and (II) ISOMAP for BA11 & PCA for BA47 to preserve the temporal sequence of the data while incorporating domain knowledge, eliminating the need to search for an optimal order of GE patterns across multiple circadian genes within an observable time window. Note that BA11 and BA47 are the two Brodmann Areas (BA) of the prefrontal cortex in the human brain in our experimental studies. ISOMAP and PCA are the selected DR approaches for BA11 and BA47, respectively, after our experiments. Specifically, drawing inspiration from sliding time windows for sequential data analysis, we replace CNNs with AutoEncoders. Instead of collectively processing numerous large matrices of GE patterns across multiple circadian genes within an observable time window, AutoEn-

coders can learn each GE pattern individually to extract the key feature from it.

3. We perform an extensive training and fine-tuning process on 16 ML models across five single-, eight ensemble-, and three deep learning-based regressors to identify the most effective ML model, i.e., ExtraTrees Regressor for BA11 and AdaBoost Regressor for BA47, respectively. Each model is evaluated under equalized execution conditions and assessed using five performance metrics: Mean Squared Error (MSE), Mean Absolute Error (MAE), Residual Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and symmetrical Mean Absolute Percentage Error (sMAPE).
4. We conduct a comprehensive experimental study and analysis on 146 subjects from (Chen et al., 2016). For each subject, BA11 and BA47 with 235 circadian gene's expression patterns (Chen et al., 2016) are examined across the five metrics above. Our approach outperforms both non-temporal-encoding-based and temporal-encoding-based models to achieve the lowest hourly-scaled MAE (0.839), MSE (1.013), and RMSE (1.006) in BA11, as well as MAE (1.227), MSE (2.153), and RMSE (1.467) in BA47, respectively. In short, our approach can predict TODs in each ZT bin at most one-hour error in BA11 and two-hour error in BA47.

The rest of this paper is structured as follows: Section 2 provides an overview of our proposed DC pipeline. Section 3 explains our two-stage DR approach. Section 4 describes our model selection and optimization. Section 5 presents an extensive experimental evaluations and discussions on TOD predictions of BA11 and BA47. Section 6 concludes the paper and briefly summarize our future work.

2 OUR PROPOSED DC PIPELINE

Figure 1 illustrates our proposed Data-driven, Clinically domain-guided (DC) pipeline, which consists of seven stages: (1) Get Data, (2) Divide Data, (3) Train/Test Split, (4) Normalize Data, (5) Encode Windows, (6) Reduce Dimensionality, and (7) Train Regressors.

Get Data. We use data from (Chen et al., 2016), which examines the impact of aging on circadian GE levels in the human prefrontal cortex shown in Figure 2. This data consists of GEs from 146 subjects. The subjects' population had a mean age of 50.7 years, 75%/25% male/female, and 85%/15% Caucasians/Others. For each subject, BA11 and BA47 are sampled. That results in total 292 samples. We only examine the GEs that are identified as circadian genes. That is, there are 235 circadian genes out of 20,000 genes in each subject. We use these GEs along with their age and gender as the input data for building our models. The tabular data shown contains a number of records. Each record is a subject's TOD information with Age, Sex, Brodmann Area (BA11/BA47), and 235 circadian GE values.

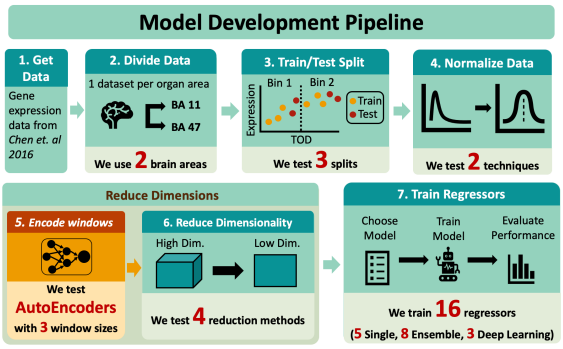

Figure 1: Overview of the Proposed DC Pipeline.

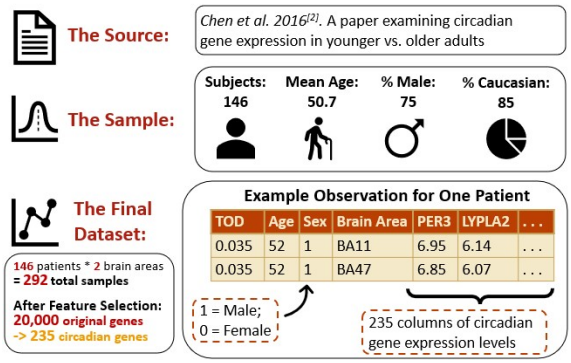

Figure 2: Data Provenance and Sampling Schematic.

Divide Data. We partition our data into two distinct datasets, one corresponding to BA11 and the other to BA47, to develop ML models tailored to each region, given their differing GE profiles. Subsequently, we sort each record in each dataset chronologically according to its TOD value in ascending order, thereby reconstructing the sequential patterns of 235 circadian GE values that evolve over time. Finally, we divide each dataset into 12 ZT bins, each spanning a two-hour time period, to facilitate further analysis and modeling.

Train/Test Split. We utilize a time-bin-based sampling approach in each ZT, where the initial T percent of data in each bin is designated for training, and the remaining data is allocated for testing, for $T \in \{60, 70, 80\}$. Given the limited sample size and

uneven distribution of observations across the 24-hour cycle, with a notable concentration between 10:00 and 22:00, this methodology ensures that our model is trained on a representative subset of data spanning the entire circadian cycle. While this approach may introduce a minor bias towards training data allocation, it facilitates the model's ability to discern underlying patterns and rhythmic fluctuations within the data.

Normalize Data. We normalize our data prior to dimensionality reduction. The purpose is to mitigate the impact of outliers and skewness, as well as optimize the performance of ML models. We evaluate two normalization techniques: Min-Max scaling and Logarithmic transformation. Our experimental results indicate that Min-Max scaling outperforms Logarithmic transformation. The Min-Max scaling method is implemented using the following formula:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}},$$

where x' denotes the normalized GE value for a specific gene at a given TOD, x represents the pre-normalized GE value, and $x_{\min}$ and $x_{\max}$ correspond to the minimum and maximum GE values observed for the same gene in the training dataset, respectively.

Encode Windows. We utilize an AutoEncoder in the first stage to compress sequences of GE values within each observable time window into a one-dimensional latent space representation, evaluating three distinct time window sizes. This latent representation effectively captures the key features of the windowed data, encoding both individual GE values and their contextual relationships with neighboring values within the window. As a result, the latent space representation encapsulates essential information regarding the curve slope and pattern changes inherent to the sinusoidal dynamics of circadian GEs.

Reduce Dimensionality. We proceed to the second stage of dimensionality reduction. With 235 latent space values, combined with Age and Gender, our dataset comprises 237 input features in total. Given the limited number of records, our training set is characterized as high-dimensional, low-sample-size data. To mitigate the curse of dimensionality, we employ a two-pronged approach, evaluating two linear dimensionality reduction techniques (Principal Component Analysis (PCA) and Independent Component Analysis (ICA)) and two non-linear methods (Kernel PCA and ISOMAP).

Train Regressors. We comprehensively train, fine-tune, and evaluate 16 ML models, encompassing five single-model regressors, eight ensemble-based regressors, and three deep learning-based regressors, to determine the most effective model for BA11 and BA47, respectively. The single-model regressors include Linear Regressor, Support Vector Regressor, Decision Tree Regressor, K-Neighbors Regressor, and Stochastic Gradient Descent Regressor. The ensemble-based models comprise Voting Regressor, Stacking Regressor, Gradient Boosting Regressor, Random Forest Regressor, AdaBoost Regressor, Bagging Regressor, ExtraTrees Regressor, and XGBoost Regressor. The deep learning-based models consist of MultiLayer Perceptron Neural Network, LSTM Network, and CNN.

3 TWO-STAGE DIMENSIONALITY REDUCTION APPROACH

3.1 First Stage Reduction

Our two-stage dimensionality reduction (DR) process comprises two key components: autoencoding and second-level feature extraction.The autoencoding process involves a three-step procedure, as illustrated in Figure 3.

In **STEP 1**, we initiate our analysis by sorting the 235 circadian GEs of BA11/BA47 in ascending order based on their corresponding TODs, thereby constructing a pseudo-multivariate time series. This time series is then segmented into multiple time windows using a sliding window approach, where each time window encompasses a sequence of consecutive GE values for a target TOD. The window size is defined as 2W + 1, where W denotes the number of GE values on either side of the target TOD. To determine the optimal window size for predicting TODs using ML models, we conduct experiments with different W values (W = 1, 2, and 3), resulting in 146 - 2W TODs for each experiment. For instance, with W = 1 and Gene1, the feature pattern vectors for consecutive target TODs would be constructed as follows: for TOD = 2, the vector would be [3.28, 8.10, 1.33]; for TOD = 3, the vector would be [8.10, 1.33, 6.74]; and for TOD = 4, the vector would be [1.33, 6.74, 9.02]. This process is applied uniformly across all 235 circadian GEs, yielding 235 GE feature pattern vectors for each target TOD. We repeat this procedure for each consecutive target TOD, spanning from TOD = 2 to TOD = 145.

In **STEP 2**, for each W, we input each feature pattern vector into the trained AutoEncoder's Encoder, yielding a one-dimensional latent space representation of the corresponding windowed sequence. Using W = 1 as an example, we obtain 144 TODs, each com-

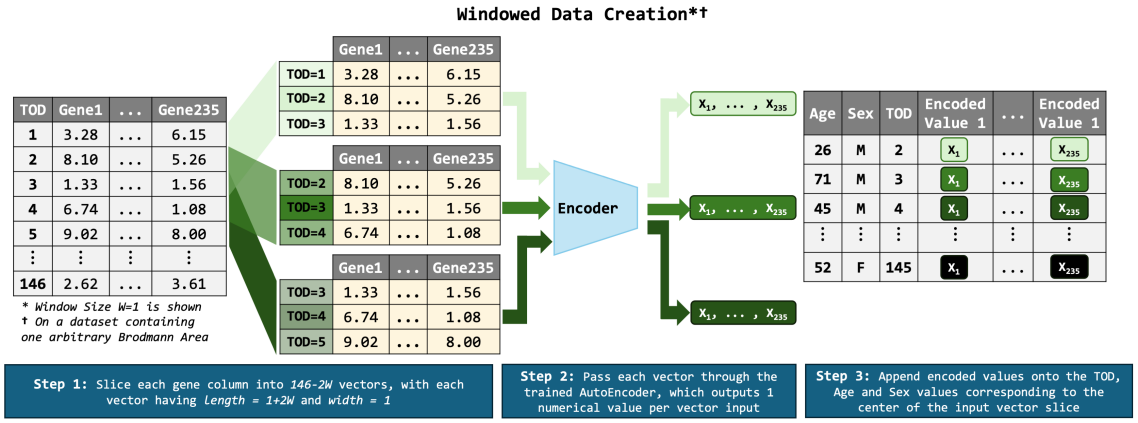

Figure 3: Autoencoding Process for Windowed Circadian Gene Expression Patterns.

prising 235 feature pattern vectors that are encoded into 235 one-dimensional latent space values, denoted as $x_1, x_1, ..., x_{235}$.

In **STEP 3**, for each target TOD record, we concatenate the 235 encoded latent space values ($[x_1, x_1, ..., x_{235}]$) with the Age, Sex, and TOD columns to finalize our dataset. For example, for TOD=2, $[x_1, x_1, ..., x_{235}]$, 26, and M are the input features.

Specifically, the structure of our AutoEncoder is shown in Figure 4 that is an encoder-decoder architecture trained via unsupervised learning to reconstruct its own input data. Specifically, in our case, it encodes each feature pattern vector within a time window (2W + 1) into one-dimensional latent space value, which is then decoded into a reconstructed feature pattern vector. The AutoEncoder measures its reconstructed feature pattern vectors against the original input themselves and then iteratively adjusts its weights of the network until the error between the reconstructed and the original feature pattern vector is minimized.

The Encoder consists of four layers with 128, 64, 32, and 1 neurons, respectively. The first three layers employ the rectified linear unit (ReLU) activation function, $f(x) = \max(0, x)$, which maps all negative inputs x to 0 while preserving positive values x unchanged. The final encoding layer uses the hyperbolic tangent (Tanh) activation function,

$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1,$$

which maps inputs to the interval $[-1, 1]$. The Tanh function exhibits a steep slope, resulting in outputs that tend to be strongly polarized, typically falling near the extremes of -1 or 1. The final encoded value is then passed through the Decoder. Equation (1) illustrates the Encoder's architecture.

Encoder:

$$\begin{aligned} h_1 &= \text{ReLU}\left(W_1^{(E)} x + b_1^{(E)}\right), \\ h_2 &= \text{ReLU}\left(W_2^{(E)} h_1 + b_2^{(E)}\right), \\ h_3 &= \text{ReLU}\left(W_3^{(E)} h_2 + b_3^{(E)}\right), \\ z &= \text{Tanh}\left(W_4^{(E)} h_3 + b_4^{(E)}\right), \end{aligned} \tag{1}$$

where

$W_1^{(E)} \in \mathbb{R}^{128\times d}$, $b_1^{(E)} \in \mathbb{R}^{128}$, $W_2^{(E)} \in \mathbb{R}^{64\times 128}$, $b_2^{(E)} \in \mathbb{R}^{64}$, $W_3^{(E)} \in \mathbb{R}^{32\times 64}$, $b_3^{(E)} \in \mathbb{R}^{32}$, $W_4^{(E)} \in \mathbb{R}^{1\times 32}$, $b_4^{(E)} \in \mathbb{R}^1$.

The Decoder consists of three decoding layers with 32, 64, and 128 neurons, respectively. The Decoder reconstructs the feature pattern vector, allowing for the computation of reconstruction error loss relative to the original input. Each neuron in the Decoder utilizes the ReLU activation function. Equation (2) illustrates the Decoder's architecture.

Decoder:

$$\begin{aligned} h_3' &= \text{ReLU}\left(W_4^{(D)} z + b_4^{(D)}\right), \\ h_2' &= \text{ReLU}\left(W_3^{(D)} h_3' + b_3^{(D)}\right), \\ h_1' &= \text{ReLU}\left(W_2^{(D)} h_2' + b_2^{(D)}\right), \\ \hat{x} &= W_1^{(D)} h_1' + b_1^{(D)}, \end{aligned} \tag{2}$$

where

$W_4^{(D)} \in \mathbb{R}^{32\times 1}$, $b_4^{(D)} \in \mathbb{R}^{32}$, $W_3^{(D)} \in \mathbb{R}^{64\times 32}$, $b_3^{(D)} \in \mathbb{R}^{64}$, $W_2^{(D)} \in \mathbb{R}^{128\times 64}$, $b_2^{(D)} \in \mathbb{R}^{128}$, $W_1^{(D)} \in \mathbb{R}^{d\times 128}$, $b_1^{(D)} \in \mathbb{R}^d$.

3.2 Second Stage Reduction

Following the first dimensionality reduction stage, the 235 encoded values, Age, and Gender are fed into the

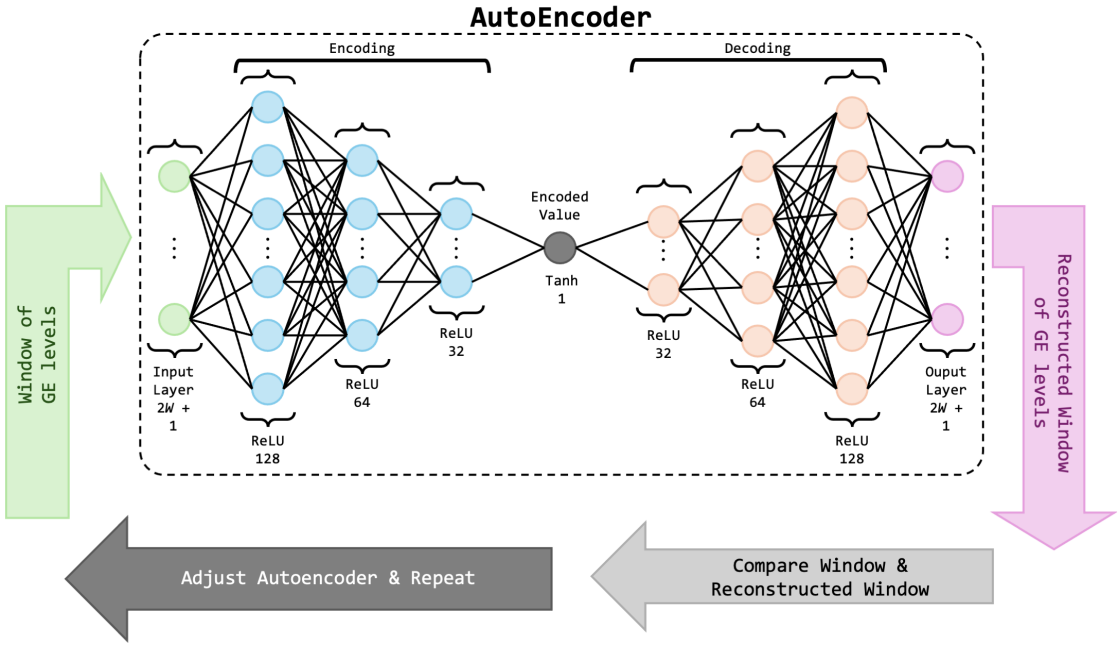

Figure 4: AutoEncoder Architecture Used for Encoding Windowed Gene Expression Patterns.

second stage for further reduction. In this stage, we utilize ISOMAP for BA11 and PCA for BA47, with detailed descriptions and illustrated examples of each method's calculations presented below.

3.2.1 ISOMAP

Let $\mathbf{X} \in \mathbb{R}^{n \times p}$ be the data matrix output from the AutoEncoder, where n is the number of samples, and p is the number of features. For simplicity, we consider a small example, where $n = 3$ and $p = 2$. Our data matrix $\mathbf{X}$ is given by:

$$\mathbf{X} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

STEP 1. Construct a neighborhood graph by using the K-nearest neighbors approach with $K = 2$. The Euclidean distance between two points $\mathbf{x}_i = (x_{i1}, x_{i2})$ and $\mathbf{x}_j = (x_{j1}, x_{j2})$ is given by:

$$d_{ij} = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2}$$

In our example:

$$d_{12} = \sqrt{(1-3)^2 + (2-4)^2} = \sqrt{8}$$

$$d_{13} = \sqrt{(1-5)^2 + (2-6)^2} = \sqrt{32}$$

$$d_{23} = \sqrt{(3-5)^2 + (4-6)^2} = \sqrt{8}$$

Based on the K-nearest neighbors rule with $K = 2$, we connect each point to its two closest neighbors. So, we have edges between (1, 2), (2, 3), and (1, 3) with the corresponding distances as edge weights.

STEP 2. Compute geodesic distances for the shortest path between the vertices of the graph. Since our graph is quite simple here and there are no alternative paths to consider for shortcuts in this small example, the geodesic distances are the same as the direct distances between the neighboring points:

$$d_{12}^{geo} = \sqrt{8}$$

$$d_{13}^{geo} = \sqrt{32}$$

$$d_{23}^{geo} = \sqrt{8}$$

STEP 3. Apply multidimensional scaling (MDS) to translate the distances between each pair of vertexes in a graph into a configuration of points mapped into an abstract Cartesian space.

1. **Construct the matrix** D^2: Each element $D_{ij}^2 = (d_{ij}^{geo})^2$. So we have:

$$D^2 = \begin{pmatrix} 0 & 8 & 32 \\ 8 & 0 & 8 \\ 32 & 8 & 0 \end{pmatrix}$$

2. **Compute the centering matrix** J: For $n = 3$, the centering matrix $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T$, where I is the $n \times n$ identity matrix and $\mathbf{1}$ is a $n \times 1$ vector of all ones.

$$J = \begin{pmatrix} \frac{2}{3} & -\frac{1}{3} & -\frac{1}{3} \\ -\frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ -\frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \end{pmatrix}$$

3. **Compute the centered matrix** B:

$$B = -\frac{1}{2} J D^2 J$$

$$B = \begin{pmatrix} 8 & 0 & -8 \\ 0 & 0 & 0 \\ -8 & 0 & 8 \end{pmatrix}$$

STEP 4. Find the eigenvalues λ_i and eigenvectors $\mathbf{v}_i$ such that $B\mathbf{v}_i = \lambda_i \mathbf{v}_i$. For our matrix B, through the standard eigenvalue calculation methods, we find the eigenvalues: $\lambda_1 = 16$, $\lambda_2 = 0$, and $\lambda_3 = 0$. The corresponding eigenvectors (up to scalar multiples) are:

$$\mathbf{v}_1 = \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix}, \mathbf{v}_2 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \mathbf{v}_3 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

STEP 5. Embed our data into a one-dimensional space by choosing $k = 1$. We select the eigenvector corresponding to the largest eigenvalue. Let's choose $\mathbf{V}_1 = [\mathbf{v}_1]$. The low-dimensional embedding $\mathbf{Y}$ is given by:

$$\mathbf{Y} = \mathbf{V}_1^T D$$

First, we need to take the square roots of the elements in D^2 to get D:

$$D = \begin{pmatrix} 0 & \sqrt{8} & \sqrt{32} \\ \sqrt{8} & 0 & \sqrt{8} \\ \sqrt{32} & \sqrt{8} & 0 \end{pmatrix}$$

$$\mathbf{Y} = \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix}^T \begin{pmatrix} 0 & \sqrt{8} & \sqrt{32} \\ \sqrt{8} & 0 & \sqrt{8} \\ \sqrt{32} & \sqrt{8} & 0 \end{pmatrix}$$

$$\mathbf{Y} = \begin{pmatrix} \sqrt{32} & 0 & -\sqrt{32} \end{pmatrix}$$

This gives us the one-dimensional embedding of our three original data points. If we are aiming for a two-dimensional embedding, we choose $K = 2$ and use two appropriate eigenvectors to form the embedding matrix and perform the calculation in a similar way. In our experiments, applying ISOMAP to BA11 effectively reduced the dataset's dimensionality that retains 90% data variance, preserving the target TODs for our ML model training, fine-tuning, and testing.

3.2.2 PCA

Let $\mathbf{X} \in \mathbb{R}^{n \times p}$ be the data matrix output from the AutoEncoder where n is the number of samples, and p is the number of features. For simplicity, we consider a small example where $n = 3$ and $p = 2$. Our data matrix $\mathbf{X}$ is given by:

$$\mathbf{X} = \begin{pmatrix} 2 & 3 \\ 4 & 5 \\ 6 & 7 \end{pmatrix}$$

STEP 1. Compute the mean vector $\mu = \mathbb{R}^p$. In our example, the mean vector $\mu = <\mu_1, \mu_2>$, where $p = 2$ and

$$\mu_1 = \frac{2+4+6}{3} = 4$$

$$\mu_2 = \frac{3+5+7}{3} = 5$$

The centered data matrix $\mathbf{X}_c$ is then:

$$(\mathbf{X}_c)_{ij} = x_{ij} - \mu_j,$$

where

$$\mathbf{X}_c = \begin{pmatrix} 2-4 & 3-5 \\ 4-4 & 5-5 \\ 6-4 & 7-5 \end{pmatrix} = \begin{pmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{pmatrix}$$

STEP 2. Calculate the covariance matrix $\mathbf{S} \in \mathbb{R}^{p \times p}$ using $\mathbf{X}_c$:

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}_c^T \mathbf{X}_c$$

$$\mathbf{X}_c^T = \begin{pmatrix} -2 & 0 & 2 \\ -2 & 0 & 2 \end{pmatrix}$$

$$\mathbf{S} = \frac{1}{3-1} \begin{pmatrix} -2 & 0 & 2 \\ -2 & 0 & 2 \end{pmatrix} \begin{pmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix}$$

STEP 3. Solve for eigenvectors $\mathbf{v}_i \in \mathbb{R}^p$ and eigenvalues $\lambda_i \in \mathbb{R}$ such that:

$$\mathbf{S}\mathbf{v}_i = \lambda_i \mathbf{v}_i$$

For our $\mathbf{S}$, the eigenvalues are $\lambda_1 = 8$ and $\lambda_2 = 0$ by using the standard calculation method. The corresponding eigenvectors can be (up to scalar multiple):

$$\mathbf{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \mathbf{v}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

STEP 4. Project onto PCs by selecting $k < p$ principal components. Let's say $k = 1$ and $\mathbf{V}_1 = [\mathbf{v}_1]$. The projected data $\mathbf{Y} \in \mathbb{R}^{n \times k}$ is:

$$\mathbf{Y} = \mathbf{X}_c \mathbf{V}_1 = \begin{pmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -4 \\ 0 \\ 4 \end{pmatrix}$$

By applying PCA to BA47 in our experiments, we have successfully reduced the dimensionality of the datasets that retains 95% data variance, preserving the target TODs for training, fine-tuning, and testing of ML models.

4 MODEL SELECTION AND OPTIMIZATION

To select the optimal ML models for BA11 and BA47, we conduct rigorous hyperparameter tuning followed by comprehensive testing. This process ensures the selection of models that deliver the best performance. Below is a detailed overview of the methodology employed.

4.1 Hyperparameter Tuning

We utilize Random Search with 5-fold Cross-Validation to explore a broad range of parameter combinations for each ML model. This approach enables the identification of optimal settings that maximize model performance across diverse subsets of the training data, ultimately enhancing generalizability and robustness.

The five single-based models with their hyperparameters include:

1. **Linear Regressor:** Default.
2. **Support Vector Regressor:** {'kernel', 'gamma', 'C'}.
3. **Decision Tree Regressor:** { 'splitter', 'min_samples_split', 'min_samples_leaf', 'min_impurity_decrease', 'max_leaf_nodes', 'max_features', 'max_depth', 'criterion', 'ccp_alpha' }.
4. **K-Neighbors Regressor:** {'weights': 'distance', 'n_neighbors'}.
5. **Stochastic Gradient Descent Regressor:** {'tol', 'power_t', 'penalty', 'max_iter', 'loss', 'learning_rate', 'l1_ratio', 'fit_intercept', 'eta0', 'alpha'}.

The eight ensemble-based models with their hyperparameters are:

1. **Voting Regressor:** { 'rf__n_estimators', 'gb__n_estimators' }
2. **Stacking Regressor:** { 'final_estimator' }
3. **Gradient Boosting Regressor:** { 'n_estimators', 'min_samples_split', 'min_samples_leaf', 'min_impurity_decrease', 'max_leaf_nodes', 'max_features', 'max_depth', 'learning_rate' }
4. **Random Forest Regressor:** { 'n_estimators', 'min_samples_split', 'min_samples_leaf', 'max_depth' }
5. **AdaBoost Regressor:** { 'n_estimators', 'loss', 'learning_rate' }
6. **Bagging Regressor:** { 'n_estimators', 'max_samples', 'max_features' }
7. **Extra Trees Regressor:** { 'n_estimators', 'min_samples_split', 'min_samples_leaf', 'max_depth' }
8. **XGBoost Regressor:** { 'subsample', 'reg_lambda', 'reg_alpha', 'n_estimators', 'min_child_weight', 'max_depth', 'learning_rate', 'gamma', 'colsample_bytree' }

The three deep learning-based models and their corresponding hyperparameters are:

1. **Multilayer Perceptron:** { 'module__num_units', 'module__activation_func', 'lr' }
2. **Long Short-Term Memory Network:** { 'module__num_layers', 'module__hidden_size', 'max_epochs', 'lr', 'batch_size' }
3. **Convolutional Neural Network:** Default

4.2 Performance Metrics for Model Evaluation

Once the optimal hyperparameters for each model are determined during training and fine-tuning, we utilize them to construct the final selected model. This step is critical, as it enables the model to fully leverage the optimal settings and adapt to the unique characteristics of our data. The final model is then evaluated on a separate testing set, where we record key performance metrics to assess its predictive accuracy and error rates. The metrics used for evaluation include:

1. **MSE (Mean Squared Error)**: Measure the average squared difference between the predicted values ($\hat{y}_i$) and the actual values (y_i) in a dataset.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

2. **MAE (Mean Absolute Error)**: Calculate the average of the absolute differences between $\hat{y}_i$ and y_i in a dataset.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

3. **MAPE (Mean Absolute Percentage Error)**: Compute this prediction accuracy as a percentage in a dataset.

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

4. **RMSE (Root Mean Squared Error)**: Take the square root of the average squared difference between $\hat{y}_i$ and y_i in a dataset.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

5. **sMAPE (symmetric Mean Absolute Percentage Error)**: Estimate forecasting accuracy by calculating the average magnitude of errors between $\hat{y}_i$ and y_i in a dataset.

$$\text{sMAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2}$$

Algorithm 1: Model Ranking and Selection Criteria.

Input: PCMResults - All Pipeline-Combination-Model (PCM) Candidates Per BA and their Corresponding MAE, MSE, RMSE, MAPE, and SMAPE
Output: BestModel
Initialization: errorMetrics = [MAE, MSE, RMSE, MAPE, sMAPE]
Def `getTheBestModel(`*PCMResults*`)`**:**
  (1) **for** *metric in errorMetrics* **do**
    sorted = Sort *PCMResults* by *metric* in the ascending order;
    $top500PCM + metric$ = Select the top 500 PCM in *sorted*;
  end
  (2) $intersectModels \leftarrow$;
    $top500PCM_{MAE}$;
    $\cap\, top500PCM_{MSE}$;
    $\cap\, top500PCM_{RMSE}$;
    $\cap\, top500PCM_{MAPE}$;
    $\cap\, top500PCM_{sMAPE}$;
  `// Retaining only those that consistently demonstrate low error rates (i.e., at least four out of five metrics)`
  (3) *topSixModels* = Select the top six models from *intersectModel* based on MAE, MSE, RMSE, MAPE, and sMAPE;
  (4) *BestModel* = Select the best model from *topSixModels* based on MAE, MSE, RMSE, MAPE, and sMAPE;
return *BestModel*

4.3 Model Ranking and Selection Criteria

Following the methodology outlined in Sections 2 through 4.2, we construct a comprehensive set of ML models for each brain area (BA11 and BA47), comprising 730 non-temporal-encoding-based, 2304 temporal-encoding-based, and 2304 AutoEncoder-enabled models. To evaluate and select the top-performing pipeline-combination-model (PCM) candidates for each model type, we employ a multi-step selection process, shown in Algorithm 1, based on five performance metrics described in Section 4.2. Specifically, we rank each PCM candidate according to its error rate for each metric, with lower scores indicating better performance. We then select the top 500 PCM candidates with the lowest error rates for each metric. To identify the most robust models, we intersect the top-performing PCM candidates across all five metrics, retaining only those that consistently demonstrate low error rates (i.e., at least four out of five metrics). From this subset, we select the top six models and ultimately identify the best-performing model based on its overall performance.

5 EXPERIMENTAL EVALUATIONS AND DISCUSSIONS

Table 1 shows the design specifications of the best model in each methodological framework, as well as the value of each performance metric that is computed based on the predicted, normalized TOD and the normalized ground-truth values. When comparing the model using the test data from BA11, our framework's best model is the top performer across all the performance metrics in terms of MAE, MSE, RMSE, MAPE, and sMAPE. When comparing the model using the test data from BA47, our framework's model outperforms across most of the performance metrics, including MAE, MSE, and RMSE, and is the second best model in terms of MAPE and sMAPE.

Table 2 shows the same best models as in Table 1, but their predicted, normalized TODs are converted back to an hourly scale, and the error metrics are recalculated. Those metrics are now on the hourly scale as well, given a better sense of these models' precision. Our framework's best model for the test data from BA11 outperforms all other models, now just those trained on the same dataset. Our framework's best model for the test data from BA47 now outperforms all other models as well in most of the metrics except MAPE that is the second lowest among them. Figure 5 shows a plot of the ground-truth TOD values versus the predicted TOD values by the models. The models trained on autoencoded data predict much closer to the one-to-one line - representing perfect accuracy - than any of the other models. Specifically, the standard deviations (StD) of error from our framework models are 0.996 (for the best BA11 model) and 1.451 (for the best BA47 model). This is an incredibly small-scale error for such biological data, which is frequently sampled in batches as opposed to immediately after a patient dies. The mean post-mortem interval (time elapsed between death and sampling) in (Chen et al., 2016) was 17.3 hours, with the minimum interval being 4.8 hours. Given the biological deterioration that such an interval can allow, error on the scale of our results is highly promising. This suggests that circadian gene expression is still active for some time after death, or that cycle disruption was minimal

Table 1: Error Metrics from the Normalized Test Data.

BA	%Train	Norm	Window Size	DR	DR Level	Model	MAE	MSE	RMSE	MAPE	sMAPE
SOTA Method 1: Non-Temporal Encoding											
11	80	Minmax	NA	PCA	90	LSTM	0.099	0.015	0.123	0.133	13.440
47	80	Minmax	NA	PCA	90	LSTM	0.127	0.020	0.143	0.172	17.321
SOTA Method 2: Temporal Encoding via CNN											
11	70	Minmax	3	PCA	90	Bagging Regressor	0.041	0.002	0.049	0.070	7.040
47	80	Log	3	KPCA	95	AdaBoost Regressor	0.109	0.020	0.142	0.039	3.977
Our Method: Temporal Encoding via AutoEncoder											
11	80	Minmax	3	Isomap	90	ExtraTrees Regressor	0.036	0.002	0.044	0.055	5.399
47	70	Minmax	3	PCA	95	AdaBoost Regressor	0.053	0.004	0.064	0.113	9.814

enough for our model to bridge the gaps.

More specifically, Table 3 and Table 4 are the number of samples in each time bin of train and test data, respectively, for our methods, i.e., ExtraTreesRegressor for BA11 and AdaBoost Regressor for BA47 that we describe above in Table 2. Table 5 and Table 6 show the MSE and StD for the testing data that resides in each two-hour time bin, respectively. The cells filled with "-" in Table 5 are hour bins, where there were not enough sample observations in the test data. The cells filled with "-" in Table 6 have only had one or zero observation in the test data; therefore, the relevant StD cannot be computed. The key takeaways in these tables show that the MSE and StD within 2-hour interval time bins did not exceed 2 (<2), indicating that the predicted TODs were generally within their own time bins. When interpreting the model performance, it is important to note that the (Chen et al., 2016) data set is a high-dimension and low-sample-size data sets. Nonetheless, with an exceptionally low margin of error across both BA11 and BA47 in our best-performing models, we see a great potential in connecting circadian clocks with healthcare practices. Some examples include pinpointing precise timings for a large range of drugs (the range is inversely proportional to prediction error) and creating accurate gene expression profiles of healthy patients for comparison with diagnostic patients. Additionally, due to the prediction stability of our proposed DC pipeline in both BA11 and BA47, we expect that our methodology can be reliably extrapolated to other brain regions and rhythmic organs.

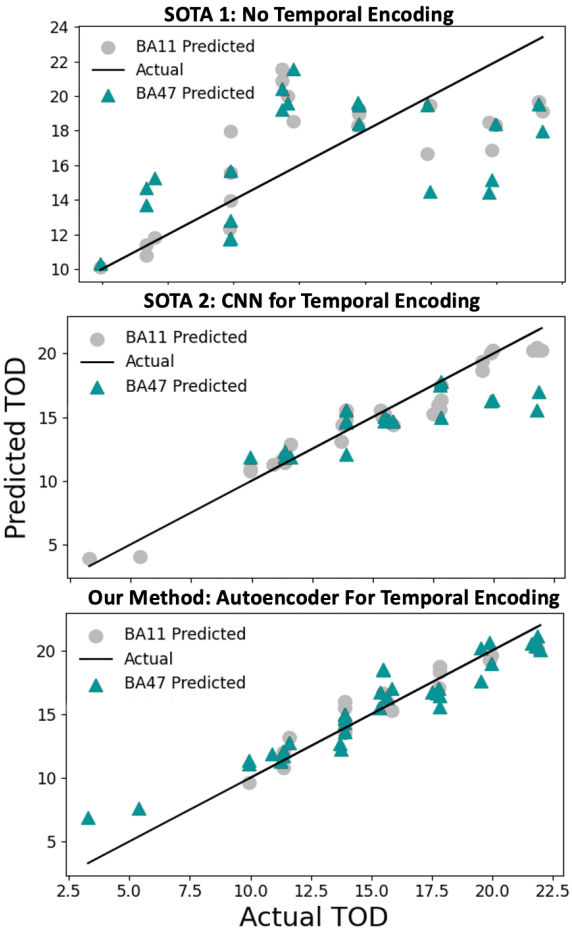

Figure 5: Actual vs. Predicted TOD of the Best-Performing Models.

6 CONCLUSIONS AND FUTURE WORK

In this work, we present a ML pipeline designed to predict the TOD of a subject based on GE levels, addressing a significant challenge in genomic research due to the lack of TOD data. Our contributions are fourfold: (1) We design a data-driven, clinically DC pipeline to learn GE patterns over time to perform TOD predictions; (2) We develop a two-stage DR approach which combines (I) AutoEncoders and (II) ISOMAP for BA11 & PCA for BA47 to preserve the temporal sequence of the data while incorporating domain knowledge, eliminating the need to search for an optimal order of GE patterns across multiple circadian genes within an observable time window; (3) We perform an extensive training and fine-tuning process on 16 ML models across five single-, eight

Table 2: Error Metrics from the Un-normalized Test Data on an Hourly Scale.

BA	%Train	Norm	Window	DR	DR Level	Model	MAE	MSE	RMSE	MAPE	sMAPE	StdErr
SOTA Method 1: Non-Temporal Encoding												
11	80	Minmax	NA	PCA	90	LSTM	2.425	9.482	3.079	0.138	13.361	3.077
47	80	Minmax	NA	PCA	90	LSTM	3.274	14.653	3.828	0.194	19.330	3.823
SOTA Method 2: Temporal Encoding via CNN												
11	70	Minmax	3	PCA	90	Bagging Regressor	0.945	1.285	1.134	0.069	7.015	1.107
47	80	Log	3	KPCA	95	AdaBoost Regressor	1.757	5.945	2.438	0.103	10.887	2.201
Our Method: Temporal Encoding via AutoEncoder												
11	80	Minmax	3	Isomap	90	ExtraTrees Regressor	0.839	1.013	1.006	0.055	5.386	0.996
47	70	Minmax	3	PCA	95	AdaBoost Regressor	1.227	2.153	1.467	0.112	9.778	1.451

Table 3: Number of Samples in Each Time Bin in Train Data.

	Bins of Hours											
	0-2	**2-4**	**4-6**	**6-8**	**8-10**	**10-12**	**12-14**	**14-16**	**16-18**	**18-20**	**20-22**	**22-24**
BA11	5	5	4	2	8	14	17	16	14	12	13	5
BA47	4	5	3	2	7	12	15	14	12	10	12	5

Table 4: Number of Samples in Each Time Bin in Test Data.

	Bins of Hours											
	0-2	**2-4**	**4-6**	**6-8**	**8-10**	**10-12**	**12-14**	**14-16**	**16-18**	**18-20**	**20-22**	**22-24**
BA11	0	0	0	0	1	3	4	4	3	2	2	0
BA47	0	1	1	0	2	5	6	6	5	4	4	0

Table 5: Mean Squared Error by Time Bins.

	Bins of Hours											
	0-2	**2-4**	**4-6**	**6-8**	**8-10**	**10-12**	**12-14**	**14-16**	**16-18**	**18-20**	**20-22**	**22-24**
BA11	-	-	-	-	0.111	1.084	1.742	0.475	0.528	0.266	2.447	-
BA47	-	12.686	4.788	-	1.686	0.571	0.857	3.657	1.870	1.406	1.862	-

Table 6: Standard Deviation of Error by Time Bins.

	Bins of Hours											
	0-2	**2-4**	**4-6**	**6-8**	**8-10**	**10-12**	**12-14**	**14-16**	**16-18**	**18-20**	**20-22**	**22-24**
BA11	-	-	-	-	-	0.885	0.921	0.646	0.683	0.126	0.195	-
BA47	-	-	-	-	0.145	0.393	0.921	1.088	0.545	1.124	0.446	-

ensemble-, and three deep learning-based regressors to identify the most effective ML model, i.e., ExtraTrees Regressor for BA11 and AdaBoost Regressor for BA47, respectively; and (4) We conduct a comprehensive experimental study and analysis on 146 subjects. For each subject, BA11 and BA47 with 235 circadian gene's expression patterns are examined across the five metrics above. Our approach outperforms both non-temporal-encoding-based and temporal-encoding-based models to achieve the lowest hourly-scaled MAE (0.839), MSE (1.013), and RMSE (1.006) in BA11, as well as MAE (1.227), MSE (2.153), and RMSE (1.467) in BA47, respectively. In short, our approach can predict TODs in each ZT bin at most one-hour error in BA11 and two-hour error in BA47. Although our ML models produce a single, deterministic TOD prediction, their inherent architecture restricts the representation of the underlying data uncertainty—a shortcoming we are actively addressing. Specifically, we are developing probabilistic machine-learning (PML) models to address this data uncertainty. These PML models will be applied to several publicly available neuropsychiatric brain gene-expression datasets to predict TOD and to incorporate rhythmicity into differential expression (DE) analyses, thereby assessing how TOD information improves DE detection.

ACKNOWLEDGMENTS

This research work is based upon work supported by the U.S. National Science Foundation under Grant

Number 2349370. Any opinions, findings, and conclusions or recommendations expressed in this work are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

REFERENCES

Anafi, R. C., Francey, L. J., Hogenesch, J. B., and Kim, J. (2017). CYCLOPS reveals human transcriptional rhythms in health and disease. *Proceedings of the National Academy of Sciences of the United States of America*, 114(20):5312–5317.

Arnold, C. D., de Souza, R. A., and Grant, P. M. (2021). Time of death estimation using postmortem gene expression data and supervised machine learning. *Journal of Forensic Sciences*, 66(3):815–825.

Blask, D. E. (2009). Melatonin, sleep disturbance and cancer risk. *Sleep Medicine Reviews*, 13(4):257–264.

Broad Institute of MIT and Harvard (2024). GTEx: Genotype-tissue expression project. Accessed 2024.

Castren, E. and Antila, H. (2017). Neuronal plasticity and neurotrophic factors in drug responses. *Molecular Psychiatry*, 22(8):1085–1095.

Chen, C.-Y., Logan, R. W., Ma, T., Lewis, D. A., Tseng, G. C., Sibille, E., and McClung, C. A. (2016). Effects of aging on circadian patterns of gene expression in the human prefrontal cortex. *Proceedings of the National Academy of Sciences*, 113(1):206–211.

Chen-Goodspeed, M. and Lee, C. C. (2007). Tumor suppression and circadian function. *Journal of Biological Rhythms*, 22(4):291–298.

Eetemadi, A. and Tagkopoulos, I. (2018). Genetic neural networks: An artificial neural network architecture for capturing gene expression relationships. *Bioinformatics*, 35(13):2226–2234.

Landgraf, D., McCarthy, M. J., and Welsh, D. K. (2014). The role of the circadian clock in animal models of mood disorders. *Behavioral Neuroscience*, 128(3):344–359.

Li, J., Cogswell, A. W., Fan, J., et al. (2018). Circadian regulation of the human transcriptome and potential time of death estimates. *Nature Communications*, 9:4639.

Li, W., Yin, Y., Quan, X., and Zhang, H. (2019). Gene expression value prediction based on XGBoost algorithm. *Frontiers in Genetics*, 10.

Liu, S., Lu, M., Li, H., and Zuo, Y. (2019). Prediction of gene expression patterns with generalized linear regression model. *Frontiers in Genetics*, 10.

Logan, R. W. and McClung, C. A. (2017). Rhythms of life: Circadian disruption and brain disorders across the lifespan. *Nature Reviews Neuroscience*, 18(8):457–471.

Marcheva, B., Ramsey, K. M., Buhr, E. D., et al. (2010). Disruption of the clock components CLOCK and BMAL1 leads to hypoinsulinaemia and diabetes. *Nature*, 466(7306):627–631.

McClung, C. A. (2013). How might circadian rhythms control mood? let me count the ways. *Biological Psychiatry*, 74(4):242–249.

Moller-Levet, C. S., Archer, S. N., Bucca, G., et al. (2013). Effects of insufficient sleep on circadian rhythmicity and expression amplitude of the human blood transcriptome. *Proceedings of the National Academy of Sciences*, 110(12):E1132–E1141.

National Center for Biotechnology Information (2024). Gene expression omnibus (GEO). Accessed 2024.

Wang, H., Li, C., Zhang, J., Wang, J., Ma, Y., and Lian, Y. (2019). A new LSTM-based gene expression prediction model: L-GEPM. *Journal of Bioinformatics and Computational Biology*, 17(4):1950022.

Yang, W., Zhang, J., and Zhao, H. (2021). Postmortem interval estimation using deep learning with RNA-seq data. *Briefings in Bioinformatics*, 22(3):1753–1764.

Yasinska-Damri, L., Babichev, S., Durnyak, B., and Goncharenko, T. (2023). Application of convolutional neural network for gene expression data classification. In Babichev, S. and Lytvynenko, V., editors, *Lecture Notes in Data Engineering, Computational Intelligence, and Decision Making*, volume 149 of *Lecture Notes on Data Engineering and Communications Technologies*. Springer, Cham.

Zhang, R., Lahens, N. F., Ballance, H. I., et al. (2014). A circadian gene expression atlas in mammals: Implications for biology and medicine. *Proceedings of the National Academy of Sciences*, 111(45):16219–16224.

Zhang, Y., Parmigiani, G., and Johnson, W. E. (2019). ComBat-seq: Batch effect adjustment for RNA-seq count data. *NAR Genomics and Bioinformatics*, 2(3):lqaa078.